Analysis of Algorithms: Lecture 3-4

MCQs with Answers · Longest Common Subsequence, Optimal Binary Search Trees

★ = **most likely to come.** All answers are taken from the Lecture 3-4 slides.

LCS: problem and definitions

1. The LCS problem is to find: ★

a) the shortest common substring
b) the length of the longest subsequence present in both sequences
c) all subsequences of one string
d) the longest substring of one string

Answer: b) the length of the longest subsequence present in both sequences

2. A subsequence appears in the same relative order, but is: ★

a) always contiguous
b) not necessarily contiguous
c) always reversed
d) always sorted

Answer: b) not necessarily contiguous

3. A string of length n has how many possible subsequences? ★

a) n
b) n^2
c) 2^n
d) n!

Answer: c) 2^n

4. How many subsequences does "abcdefg" (length 7) have?

a) 7
b) 49
c) 128
d) 5040

Answer: c) 128 ($2^7 = 128$)

5. Which is NOT a subsequence of "abcdefg"?

a) abc
b) aeg
c) acefg
d) gfa

Answer: d) gfa (Order is reversed)

6. The LCS of "ABCDGH" and "AEDFHR" is: ★

a) ADH
b) ABH
c) AEH
d) ADGH

Answer: a) ADH (Length 3)

7. The LCS of "AGGTAB" and "GXTXAYB" is: ★

a) GTAB
b) GGTA
c) AGTB
d) GXAB

Answer: a) GTAB (Length 4)

8. LCS is the basis of which program? ★

a) gcc
b) diff
c) zip
d) ping

Answer: b) diff (A file comparison program)

9. LCS has applications in:

a) bioinformatics
b) networking only
c) graphics only
d) compilers only

Answer: a) bioinformatics

10. The naive solution of LCS generates all subsequences of both strings. Its time complexity is:

a) linear
b) quadratic
c) exponential
d) logarithmic

Answer: c) exponential

LCS: DP properties

11. If the last characters of X and Y match, L(X[0..m−1], Y[0..n−1]) = ★

a) MAX(...)
b) 1 + L(X[0..m−2], Y[0..n−2])
c) L(X[0..m−2], Y[0..n−1])
d) 0

Answer: b) 1 + L(X[0..m−2], Y[0..n−2])

12. If the last characters of X and Y do NOT match, L(X, Y) = ★

a) 1 + L(...)
b) MAX(L(X[0..m−2], Y), L(X, Y[0..n−2]))
c) MIN(L(...), L(...))
d) 0

Answer: b) MAX(L(X[0..m−2], Y), L(X, Y[0..n−2]))

13. L("AGGTAB", "GXTXAYB") can be written as:

a) 1 + L("AGGTA", "GXTXAY")
b) MAX(L("AGGTA", "GXTXAYB"), L("AGGTAB", "GXTXAY"))
c) L("AGGTA", "GXTXA")
d) 0

Answer: a) 1 + L("AGGTA", "GXTXAY") (Last characters (B, B) match)

14. Since the main problem is solved using solutions to subproblems, LCS has:

a) greedy property
b) optimal substructure
c) no DP property
d) heuristic property

Answer: b) optimal substructure

15. In the recursion tree for "AXYT" and "AYZX", which subproblem is solved twice? ★

a) lcs("AX", "AY")
b) lcs("AXY", "AYZ")
c) lcs("AXYT", "AY")
d) lcs("A", "A")

Answer: b) lcs("AXY", "AYZ")

16. LCS has both properties of DP: optimal substructure and: ★

a) greedy choice
b) overlapping subproblems
c) divide and conquer
d) backtracking

Answer: b) overlapping subproblems

17. Time complexity of the naive recursive LCS in the worst case is: ★

a) O(mn)
b) $O(n^2)$
c) $O(2^n)$
d) O(n)

Answer: c) $O(2^n)$

18. The worst case of naive recursive LCS happens when: ★

a) all characters match
b) all characters of X and Y mismatch
c) strings are equal
d) one string is empty

Answer: b) all characters of X and Y mismatch (LCS length = 0)

19. Recomputation of the same subproblems in LCS is avoided by:

a) sorting
b) Memoization or Tabulation
c) hashing
d) greedy choice

Answer: b) Memoization or Tabulation

LCS: algorithm and table

20. Time complexity of the tabulated (DP) LCS algorithm is: ★

a) $O(2^n)$ b) O(mn)
c) $O(n^3)$ d) O(log n)

Answer: b) O(mn) (m and n are the string lengths)

21. The LCS-LENGTH algorithm returns:

a) only the LCS string b) only the length of LCS (with tables c and b)
c) all subsequences d) the shortest subsequence

Answer: b) only the length of LCS (with tables c and b) (The b table is used to print the LCS)

22. In LCS-LENGTH, the first row and first column of table c are filled with:

a) 1 b) 0
c) ∞ d) −1

Answer: b) 0

23. If $x_i == y_j$, then c[i, j] = ★

a) c[i−1, j] b) c[i, j−1]
c) c[i−1, j−1] + 1 d) 0

Answer: c) c[i−1, j−1] + 1

24. When characters match, the arrow stored in b[i, j] is: ★

a) ↑ b) ←
c) ↖ d) →

Answer: c) ↖

25. If $x_i \neq y_j$ and c[i−1, j] ≥ c[i, j−1], then c[i, j] = c[i−1, j] and the arrow is:

a) ↖ b) ↑
c) ← d) ↓

Answer: b) ↑

26. If $x_i \neq y_j$ and c[i−1, j] < c[i, j−1], the arrow stored is:

a) ↖ b) ↑
c) ← d) →

Answer: c) ←

27. To print the LCS, we follow the arrows starting from: ★

a) the top-left corner b) the lower right-hand corner
c) the middle d) the first row

Answer: b) the lower right-hand corner (Each ↖ is a character of the LCS)

28. The LCS of X = ABCBDAB and Y = BDCABA is: ★

a) BDAB b) BCBA
c) ABBA d) BCAB

Answer: b) BCBA

29. In the above example, the length of LCS is: ★

a) 3 b) 4
c) 5 d) 6

Answer: b) 4 (c[7, 6] = 4)

Optimal BST: problem and cost

30. The Optimal BST problem constructs a BST so that: ★

a) the tree height is maximum
b) the total cost of all searches is as small as possible
c) all keys are at the same level
d) keys are unsorted

Answer: b) the total cost of all searches is as small as possible

31. In Optimal BST, the input keys array must be: ★

a) unsorted
b) sorted
c) reversed
d) random

Answer: b) sorted

32. freq[i] in Optimal BST means:

a) the value of key i
b) the number of searches to keys[i]
c) the level of key i
d) the size of the tree

Answer: b) the number of searches to keys[i]

33. The cost of a BST node is: ★

a) level + frequency
b) level × frequency
c) frequency only
d) level only

Answer: b) level × frequency

34. The level of the root in a BST is taken as: ★

a) 0
b) 1
c) 2
d) n

Answer: b) 1

35. keys = {10, 12}, freq = {34, 50}. The cost of the tree with root 10 is: ★

a) 118
b) 134
c) 84
d) 168

Answer: b) 134 (34×1 + 50×2)

36. For the same keys, the cost of the tree with root 12 is: ★

a) 118
b) 134
c) 100
d) 150

Answer: a) 118 (50×1 + 34×2 (minimum))

37. How many different BSTs are possible with 3 keys?

a) 3
b) 4
c) 5
d) 6

Answer: c) 5

38. keys = {10, 12, 20}, freq = {34, 8, 50}. The minimum cost is: ★

a) 158
b) 142
c) 168
d) 200

Answer: b) 142 (50×1 + 34×2 + 8×3)

39. In the above example, which key is the root of the optimal BST?

a) 10
b) 12
c) 20
d) none

Answer: c) 20 (Highest frequency (50) at the root)

Optimal BST: DP

40. To find the result of Optimal BST, we calculate: ★

a) optCost(1, n)
b) optCost(0, n−1)
c) optCost(0, 0)
d) optCost(n, n)

Answer: b) optCost(0, n−1)

41. In the optCost formula, the sum of frequencies from i to j is added because: ★

a) it makes the tree balanced
b) every search goes through the root and one comparison is done for every search
c) it sorts the keys
d) it reduces the height

Answer: b) every search goes through the root and one comparison is done for every search

42. In optCost(i, j), r varies from i to j because:

a) we try every node as the root one by one
b) we sort the keys
c) we delete nodes
d) we count leaves

Answer: a) we try every node as the root one by one

43. Time complexity of the naive recursive Optimal BST approach is: ★

a) O(n)
b) $O(n^2)$
c) $O(n^3)$
d) Exponential

Answer: d) Exponential

44. Optimal BST has which DP properties? ★

a) only optimal substructure
b) only overlapping subproblems
c) both optimal substructure and overlapping subproblems
d) neither

Answer: c) both optimal substructure and overlapping subproblems

45. In the DP solution, which entry holds the final result?

a) cost[0][0]
b) cost[n−1][0]
c) cost[0][n−1]
d) cost[n][n]

Answer: c) cost[0][n−1]

46. In the DP table cost[][], which values are filled first?

a) the last row
b) the diagonal values cost[i][i]
c) the first column
d) random values

Answer: b) the diagonal values cost[i][i] (Then cost[i][i+1], cost[i][i+2], ...)

47. For a single key, cost[i][i] equals:

a) 0
b) 1
c) freq[i]
d) keys[i]

Answer: c) freq[i]

48. The Optimal BST DP solution uses the same idea as:

a) Binary Search
b) Matrix Chain Multiplication
c) Quick Sort
d) Fibonacci only

Answer: b) Matrix Chain Multiplication (Chain length L, j = i + L − 1)

49. Time complexity of the DP solution of Optimal BST (as given) is: ★

a) $O(n^2)$
b) $O(n^3)$
c) $O(n^4)$
d) $O(2^n)$

Answer: c) $O(n^4)$

50. It can be reduced to $O(n^3)$ by: ★

a) sorting the keys
b) pre-calculating the sum of frequencies
c) using recursion
d) removing the root

Answer: b) pre-calculating the sum of frequencies (Instead of calling sum() again and again)

51. To store the structure of the optimal BST, we store in an auxiliary array:

a) the frequencies
b) the chosen root r
c) the keys
d) the levels

Answer: b) the chosen root r (Stored in the innermost loop)

Short on time? Learn only the ★ MCQs

★ MCQ numbers: **1, 2, 3, 6, 7, 8, 11, 12, 15, 16, 17, 18, 20, 23, 24, 27, 28, 29, 30, 31, 33, 34, 35, 36, 38, 40, 41, 43, 44, 49, 50**

5 common traps in MCQs

1. **Subsequence** = same order, gaps allowed. A string of length n has 2^n subsequences.
2. **Naive LCS = $O(2^n)$** (worst when all characters mismatch). **DP LCS = $O(mn)$**.
3. Match = ↖ **diagonal + 1**. No match = ↑ if up ≥ left, otherwise ←. Print from the **lower right** corner.
4. BST cost = **level × frequency**, root level = **1**. Answers: **118** and **142**.
5. Optimal BST DP = **$O(n^4)$**, reduced to **$O(n^3)$** with pre-calculated sums. Final answer in **cost[0][n−1]**.